

AgroSearch — Motor de Busca Inteligente

UNIPÊ — Centro Universitário de João Pessoa · Tópicos Avançados · Laboratório Prático 04
Prof. Me. Ricardo Roberto de Lima · Equipe: [Integrante 1] · [Integrante 2] · [Integrante 3]

1. Objetivo

Protótipo de motor de busca textual para os manuais técnicos da AgroTech Solutions. O técnico digita uma consulta e recebe os trechos mais relevantes, ranqueados por TF-IDF. Seguindo a restrição do desafio, **nada foi feito com scikit-learn**: pré-processamento, índice invertido, TF-IDF e cosseno foram implementados do zero em Python puro (*re*, *unicodedata*, *math*, *collections*), com interface em Streamlit.

2. Arquitetura (agrosearch_app.py)

- **Fase 1 — PipelineTexto**: tokenização por regex → normalização (minúsculas e remoção de acentos via NFD) → remoção de stopwords (lista própria em português) → stemmer em quatro estágios inspirado no RSLP (plural → sufixos nominais → terminações verbais → vogal final), cada estágio removendo no máximo um sufixo e preservando um radical de 3+ letras. Tudo fica na classe *PipelineTexto*. Stopwords e stemming têm checkboxes na sidebar; com as duas etapas ligadas o vocabulário cai de 47 para 33 termos.
- **Fase 2 — ÍndiceInvertido**: mapa termo → postings list, com df e IDF por termo; exibido em tabela e em JSON.
- **Fase 3 — RanqueadorTFIDF**: $TF(t,d) = f(t,d)/|d|$; $IDF(t) = \log_{10}(N/df(t))$; $score(d) = \text{soma do TF-IDF dos termos da consulta}$. A tela abre direto no resultado: o trecho vencedor em destaque e a tabela ordenada do maior para o menor, com a linha vencedora realçada. A "memória de cálculo" mostra o cálculo termo a termo.
- **Bônus — Cosseno**: vetores TF-IDF da consulta e dos trechos sobre o vocabulário inteiro e $\cos = q \cdot d / (|q||d|)$, comparado com o TF-IDF acumulado.
- **Interface**: a consulta fica no topo e os bastidores na seção "Por dentro do motor" (pipeline, índice, memória de cálculo e cosseno). Os trechos da base podem ser editados numa tabela na sidebar.

3. Exemplo: consulta "lagartas na soja"

Após o pipeline: [lagart, soj]. $df(lagart) = 2 \rightarrow IDF = \log_{10}(5/2) = 0,398$; $df(soj) = 3 \rightarrow IDF = \log_{10}(5/3) = 0,222$.

Doc	Termo	f/ d	TF	df	IDF	TF-IDF
D1	soj	1/8	0.125	3	0.222	0.0277
D2	lagart	1/7	0.143	2	0.398	0.0568
D2	soj	1/7	0.143	3	0.222	0.0317
D4	lagart	1/8	0.125	2	0.398	0.0497
D4	soj	1/8	0.125	3	0.222	0.0277

Pos.	Documento	TF-IDF acumulado
1	D2	0.0885
2	D4	0.0775
3	D1	0.0277
4	D3	0.0000
5	D5	0.0000

O D2 vence o D4 porque é mais curto (7 × 8 tokens): o mesmo par de termos pesa mais no TF. O D1 só tem "soja", o termo menos discriminativo.

4. Observações

- Somar TF-IDF favorece documentos com **um** termo raro. No bônus, o cosseno considera a consulta inteira e penaliza documentos que tratam de muitos outros assuntos.
- O stemmer por regras junta "lagartas"/"lagarta" e "irrigação"/"irrigar", mas também produz radicais sem significado ("requ", "algoda"). É o custo esperado de um stemmer heurístico, citado em aula.
- Termos da consulta fora do vocabulário recebem IDF 0 e aparecem num aviso na interface.

5. Divisão de tarefas

Integrante	Responsabilidade
[Integrante 1]	PipelineTexto e stemmer em estágios (Fase 1)
[Integrante 2]	ÍndiceInvertido e RanqueadorTFIDF (Fases 2 e 3)
[Integrante 3]	Interface Streamlit, bônus do cosseno e relatório

Execução: *pip install streamlit pandas e streamlit run agrosearch_app.py.*